

ÉCOLE D'INGÉNIEURS SUP GALILÉE

Rapport du TP1 de Base De Données Avancées

Encadrant :
Mr YUCEF

Élève :
Maryse GOEH-AKUE
N° étudiant : 12101028

1 Exercice 1 : Création et peuplement de la base

1.1 1) Création / modification de la table section

```
CREATE TABLE section (
  course_id    VARCHAR2(8),
  sec_id       VARCHAR2(8),
  semester     VARCHAR2(6),
  year         NUMBER(4,0),
  building     VARCHAR2(15),
  room_number  VARCHAR2(7),
  time_slot_id VARCHAR2(4),
  PRIMARY KEY (course_id, sec_id, semester, year),
  CONSTRAINT chk_semester CHECK (semester IN ('Fall','Winter','Spring','Summer')),
  FOREIGN KEY (course_id) REFERENCES course(course_id),
  FOREIGN KEY (building, room_number) REFERENCES classroom(building, room_number)
);
```

FIGURE 1 – Création de la table `section` avec clé primaire, clés étrangères et contrainte CHECK sur `semester`.

1.2 2) Schéma relationnel / diagramme

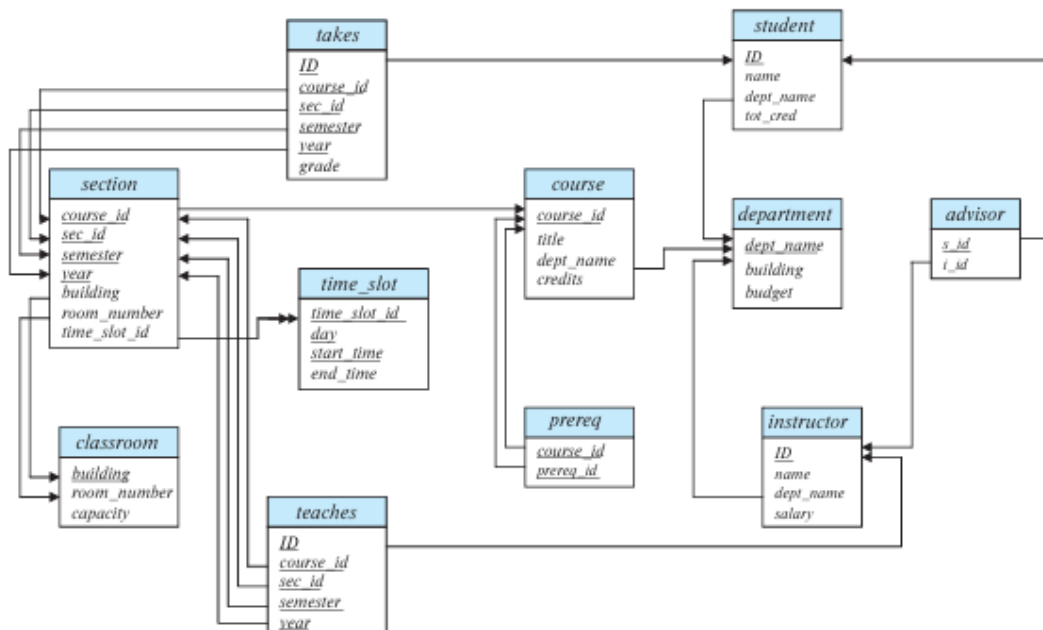


Figure 1: Schéma relationnel de la base de données de gestion des cours

FIGURE 2 – Schéma relationnel de la base (relations et contraintes).

1.3 3) Vérification du peuplement des tables

```

1  SELECT 'classroom' AS table_name, COUNT(*) AS nb FROM classroom
2  UNION ALL
3  SELECT 'department' AS table_name, COUNT(*) AS nb FROM department
4  UNION ALL
5  SELECT 'course' AS table_name, COUNT(*) AS nb FROM course
6  UNION ALL
7  SELECT 'teacher' AS table_name, COUNT(*) AS nb FROM teacher
8  UNION ALL
9  SELECT 'student' AS table_name, COUNT(*) AS nb FROM student
10 UNION ALL

```

Results	Explain	Describe	Saved SQL	History
TABLE_NAME				
classroom				
department				
course				
teacher				
student				
section				
takes				

FIGURE 3 – Vérification du chargement des données : comptage du nombre de tuples par table.

1.4 4) Insertion d'un cours supplémentaire et vérification

Dans notre base de départ, le cours BIO-101 est déjà présent. Nous ajoutons donc un cours BIO-102 afin de vérifier le bon fonctionnement de l'insertion et de l'affichage des données.

```

1  SELECT * FROM course WHERE course_id IN ('BIO-101','BIO-102');

```

COURSE_ID	TITLE	DEPT_NAME	CREDITS
BIO-101	Intro. to Biology	Biology	4
BIO-102	Biology Lab	Biology	1

2 rows returned in 0.01 seconds Download

FIGURE 4 – Insertion du cours BIO-102 et affichage de BIO-101 et BIO-102 dans la relation course.

2 Exercice 2 : Requêtes SQL (Q1 à Q24)

Q1. Afficher la structure de la relation section et son contenu

```
1 SELECT * FROM section;
```

Results	Explain	Describe	Saved SQL	History
COURSE_ID	SEC_ID	SEMESTER		
CS-315	1	Spring		
CS-319	1	Spring		
CS-319	2	Spring		
CS-347	1	Fall		

More than 10 rows available. Increase rows selector to view more rows.

FIGURE 5 – Q1 : Structure et contenu de section.

Q2. Afficher tous les renseignements sur les cours (relation course)

```
SELECT * FROM course;
```

FIGURE 6 – Q2 : Contenu de la relation course.

Q3. Afficher les titres des cours et les départements qui les proposent

```
SELECT title, dept_name  
FROM course;
```

FIGURE 7 – Q3 : Titres des cours et départements correspondants.

Q4. Afficher les noms des départements ainsi que leur budget

```
SELECT dept_name, budget  
FROM department;
```

FIGURE 8 – Q4 : Départements et budgets.

Q5. Afficher tous les noms des enseignants et leur département

```
SELECT name, dept_name  
FROM teacher;
```

FIGURE 9 – Q5 : Enseignants et départements.

Q6. Afficher les noms des enseignants ayant un salaire strictement supérieur à 65 000

```
SELECT name, salary
FROM teacher
WHERE salary > 65000;
```

FIGURE 10 – Q6 : Enseignants avec salaire > 65 000.

Q7. Afficher les noms des enseignants ayant un salaire compris entre 55 000 et 85 000

```
SELECT name, salary
FROM teacher
WHERE salary BETWEEN 55000 AND 85000;
```

FIGURE 11 – Q7 : Enseignants avec salaire entre 55 000 et 85 000.

Q8. Afficher les noms des départements (via teacher) en éliminant les doublons

```
SELECT DISTINCT dept_name
FROM teacher;
```

FIGURE 12 – Q8 : Départements distincts à partir de teacher.

Q9. Afficher les enseignants de l'informatique avec salaire $> 65\,000$

```
1 SELECT name, salary
2 FROM teacher
3 WHERE dept_name = 'Comp. Sci.'
4 | AND salary > 65000;
```

NAME	SALARY
Katz	75000
Brandt	92000

FIGURE 13 – Q9 : Enseignants du département Comp. Sci. avec salaire $> 65\,000$.**Q10. Afficher les cours proposés au Spring 2010 (relation section)**

```
1 SELECT title, credits
2 FROM course
3 WHERE dept_name = 'Comp. Sci.'
4 | AND credits > 3;
```

TITLE	CREDITS
Intro. to Computer Science	4
Game Design	4

FIGURE 14 – Q10 : Enseignements proposés au Spring 2010.

Q11. Afficher les titres des cours du département informatique avec plus de 3 crédits

```
SELECT title, credits
FROM course
WHERE dept_name = 'Comp. Sci.'
| AND credits > 3;
```

FIGURE 15 – Q11 : Cours de Comp. Sci. avec credits > 3 .

Q12. Afficher les enseignants, leur département et le bâtiment du département

```
SELECT t.name AS teacher_name, d.dept_name, d.building
FROM teacher t
JOIN department d ON d.dept_name = t.dept_name;
```

FIGURE 16 – Q12 : Enseignants, département et bâtiment associé.

Q13. Afficher les étudiants ayant suivi au moins un cours en informatique

```
SELECT DISTINCT s.name AS student_name
FROM student s
JOIN takes tk ON tk.id = s.id
JOIN course c ON c.course_id = tk.course_id
WHERE c.dept_name = 'Comp. Sci.';
```

FIGURE 17 – Q13 : Étudiants ayant suivi au moins un cours en informatique.

Q14. Afficher les étudiants ayant suivi un cours dispensé par Einstein (sans doublons)

```
SELECT DISTINCT s.name AS student_name
FROM student s
JOIN takes tk ON tk.id = s.id
JOIN teaches te
  ON te.course_id = tk.course_id
  AND te.sec_id    = tk.sec_id
  AND te.semester  = tk.semester
  AND te.year      = tk.year
JOIN teacher t ON t.id = te.id
WHERE t.name = 'Einstein';
```

FIGURE 18 – Q14 : Étudiants ayant suivi un cours enseigné par Einstein.

Q15. Afficher les identifiants des cours et les enseignants qui les ont assurés

```
SELECT DISTINCT te.course_id, t.name AS teacher_name
FROM teaches te
JOIN teacher t ON t.id = te.id
ORDER BY te.course_id, t.name;
```

FIGURE 19 – Q15 : Cours et enseignants correspondants.

Q16. Afficher le nombre d'inscrits pour chaque enseignement proposé au Spring 2010

```
SELECT s.course_id, s.sec_id, s.semester, s.year,  
       COUNT(tk.id) AS nb_inscrits  
FROM section s  
LEFT JOIN takes tk  
  ON tk.course_id = s.course_id  
 AND tk.sec_id    = s.sec_id  
 AND tk.semester  = s.semester  
 AND tk.year      = s.year  
WHERE s.semester = 'Spring'  
 AND s.year = 2010  
GROUP BY s.course_id, s.sec_id, s.semester, s.year  
ORDER BY s.course_id, s.sec_id;
```

FIGURE 20 – Q16 : Nombre d'inscrits par enseignement au Spring 2010.

Q17. Afficher les départements et le salaire maximum de leurs enseignants

```
SELECT dept_name, MAX(salary) AS max_salary  
FROM teacher  
GROUP BY dept_name  
ORDER BY dept_name;
```

FIGURE 21 – Q17 : Salaire maximum par département.

Q18. Afficher le nombre d'inscrits pour chaque enseignement proposé

```
SELECT s.course_id, s.sec_id, s.semester, s.year,
       COUNT(tk.id) AS nb_inscrits
FROM section s
LEFT JOIN takes tk
  ON tk.course_id = s.course_id
 AND tk.sec_id    = s.sec_id
 AND tk.semester  = s.semester
 AND tk.year      = s.year
GROUP BY s.course_id, s.sec_id, s.semester, s.year
ORDER BY s.year, s.semester, s.course_id, s.sec_id;
```

FIGURE 22 – Q18 : Nombre d'inscrits par enseignement (toutes périodes).

Q19. Nombre total de cours par bâtiment (Fall 2009 et Spring 2010)

```
SELECT building, COUNT(*) AS nb_cours
FROM section
WHERE (semester = 'Fall' AND year = 2009)
     OR (semester = 'Spring' AND year = 2010)
GROUP BY building
ORDER BY building;
```

FIGURE 23 – Q19 : Nombre de cours par bâtiment (Fall 2009 et Spring 2010).

Q20. Nombre total de cours par département ayant eu lieu dans son bâtiment

```
SELECT d.dept_name, COUNT(*) AS nb_cours
FROM department d
JOIN course c    ON c.dept_name = d.dept_name
JOIN section s   ON s.course_id = c.course_id
WHERE s.building = d.building
GROUP BY d.dept_name
ORDER BY d.dept_name;
```

FIGURE 24 – Q20 : Nombre de cours par département ayant eu lieu dans le bâtiment du département.

Q21. Titres des cours ayant eu lieu et enseignants qui les ont assurés

```
SELECT c.title, t.name AS teacher_name,
       s.course_id, s.sec_id, s.semester, s.year
FROM course c
JOIN section s  ON s.course_id = c.course_id
JOIN teaches te ON te.course_id = s.course_id
              AND te.sec_id    = s.sec_id
              AND te.semester  = s.semester
              AND te.year      = s.year
JOIN teacher t ON t.id = te.id
ORDER BY c.title, s.year, s.semester;
```

FIGURE 25 – Q21 : Cours et enseignants correspondants.

Q22. Nombre total de cours pour Summer, Fall et Spring

```
SELECT semester, COUNT(*) AS nb_cours
FROM section
WHERE semester IN ('Summer', 'Fall', 'Spring')
GROUP BY semester
ORDER BY semester;
```

FIGURE 26 – Q22 : Nombre total de cours par semestre (Summer, Fall, Spring).

Q23. Pour chaque étudiant, total de crédits obtenus hors de son département

```
SELECT student.name, sum(credits)
FROM student, course, takes
WHERE student.id = takes.id and takes.course_id = course.
course_id and student.dept_name != course.dept_name GROUP BY student.name;
```

FIGURE 27 – Q23 : Somme des crédits suivis par chaque étudiant hors de son département.

Q24. Pour chaque bâtiment, total de crédits des cours ayant eu lieu

```
SELECT section.building, sum(course.credits) FROM section, course
WHERE section.course_id = course.course_id GROUP BY section.building;
```

FIGURE 28 – Q24 : Somme des crédits par bâtiment (selon les cours ayant eu lieu).